

Desafio Técnico Looqbox

Analista de Dados e BI Pleno · Python e SQL · Remoto

Witor Sather

satherws@gmail.com

Goiânia, GO

15 de agosto de 2026

Banco: MySQL, schema **loobox-challenge**

1. Como eu organizei o trabalho

Dividi o desafio em seis etapas executadas em sequência: preparar o ambiente e validar a conexão, mapear o schema antes de escrever qualquer query, responder as três perguntas de SQL, construir a função dinâmica do Case 1, reproduzir o TM do Case 2 e, por último, criar a visualização própria do Case 3. Cada etapa virou um script separado que grava a própria saída em arquivo, então todo número que aparece neste documento pode ser reproduzido rodando o script correspondente.

Arquivo	O que faz
01b_schema.py	Conecta, descobre o schema e mapeia tabelas, tipos, volume e período
02_sql_test.py	Responde as três perguntas do SQL Test, com diagnóstico e controle
loobox_data.py	Módulo do Case 1, com a função <code>retrieve_data</code>
03_case1_demo.py	Dez cenários de teste da função, incluindo os que devem falhar
04_case2_tm.py	Case 2: queries do cliente, filtro no Python, TM e gráfico
05_case3_imdb.py	Case 3: perfil do IMDB e as duas visualizações

1.1. Primeiro obstáculo: o nome do schema

A primeira tentativa de conexão falhou com o erro 1044. O e-mail do processo indica o schema como `loobox-challenge`, com hífen, e o README do repositório indica `loobox_challenge`, com underline. Como o erro 1044 é negativa de acesso a um banco específico, e não credencial inválida, que seria 1045, conectei sem definir schema e pedi a lista ao próprio servidor.

```
pymysql.err.OperationalError: (1044, "Access denied for user  
'loobox-challenge'@ '%' to database 'loobox_challenge'")  
  
>>> Conectando SEM schema definido ...  
SCHEMAS VISIVEIS: ['information_schema', 'loobox-challenge', 'performance_schema']
```

O schema correto é **loobox-challenge**, com hífen. Por causa do hífen, o nome precisa vir entre crases nas queries e escapado na URL de conexão.

1.2. O que o mapeamento mostrou

Tabela	Linhas	Período	Observação relevante
<code>data_product</code>	9.994	-	Preço em <code>PRODUCT_VAL</code> , decimal(8,2)
<code>data_product_sales</code>	2.173.133	23/11/2014 a 31/12/2019	STORE_CODE é varchar(255)
<code>data_store_cad</code>	20	-	STORE_CODE é int , datas como text
<code>data_store_sales</code>	35.762	23/11/2014 a 31/12/2019	Venda agregada da loja
<code>IMDB_movies</code>	1.000	filmes de 2006 a 2016	Rating e Revenue como decimal(10,0)

Ponto de atenção que guiou todo o resto: a mesma informação, o código da loja, está como **texto** em `data_product_sales` e como **número** em `data_store_cad`. Qualquer join entre as duas sem tratar isso corre o risco de descartar linhas em silêncio, que é o pior tipo de erro em análise de dados, porque o relatório fica pronto, bonito e errado.

2. SQL Test

2.1. Quais são os 10 produtos mais caros da empresa

O preço de tabela do produto está em `data_product.PRODUCT_VAL`. Não usei `data_product_sales` porque lá o valor é a venda do dia, e não o preço unitário.

```
SELECT
    PRODUCT_COD,
    PRODUCT_NAME,
    PRODUCT_VAL,
    DEP_NAME,
    SECTION_NAME
FROM data_product
ORDER BY PRODUCT_VAL DESC
LIMIT 10
```

Cód.	Produto	Preço (R\$)	Departamento	Seção
301409	Whisky Escocês THE MACALLAN Ruby Garrafa 700ml com Caixa	741,99	BEBIDAS	BEBIDAS
176185	Whisky Escocês JOHNNIE WALKER Blue Label Garrafa 750ml	735,90	BEBIDAS	BEBIDAS
315481	Cafeteira Expresso 3 CORAÇÕES Tres Modo Vermelho	499,00	BEBIDAS	BEBIDAS
100280	Vinho Português Tinto Vintage QUINTA DO CRASTO Garrafa 750ml	445,90	BEBIDAS	VINHOS
320046	Escova Dental Elétrica ORAL B D34 Professional Care 5000 110v	399,90	PERFUMARIA	HIGIENE BUCAL
190817	Champagne Rosé VEUVE CLICQUOT PONSARDIM Garrafa 750ml	366,90	MERCEARIA	ARTIGOS-PARA-O-LAR
153795	Champagne Francês Brut Imperial MOET Rosé Garrafa 750ml	359,90	MERCEARIA	ARTIGOS-PARA-O-LAR
311397	Conjunto de Panelas Allegra em Inox TRAMONTINA 5 Peças	359,00	MERCEARIA	ARTIGOS-PARA-O-LAR
147706	Whisky Escocês CHIVAS REGAL 18 Anos Garrafa 750ml	329,90	BEBIDAS	BEBIDAS
154431	Champagne Francês Brut Imperial MOET & CHANDON Garrafa 750ml	315,90	MERCEARIA	ARTIGOS-PARA-O-LAR

Empate na fronteira. Rodei uma checagem extra nos preços da faixa do top 10 e o valor R\$ 315,90 aparece em **dois** produtos. Como ele é exatamente o preço da décima posição, existe um décimo primeiro produto empatado que o `LIMIT 10` corta. Se a intenção do negócio for "todos os produtos que estão entre os dez maiores preços", a consulta correta usa função de janela:

```
SELECT * FROM (
  SELECT PRODUCT_COD, PRODUCT_NAME, PRODUCT_VAL, DEP_NAME, SECTION_NAME,
    DENSE_RANK() OVER (ORDER BY PRODUCT_VAL DESC) AS POSICAO
  FROM data_product
) t
WHERE POSICAO <= 10
```

Leitura: seis dos dez itens são bebida alcoólica premium e quatro deles são whisky ou champagne. O topo da tabela de preços é dominado por presente e celebração, não por item de cesta básica.

2.2. Quais seções existem nos departamentos BEBIDAS e PADARIA

A granularidade da tabela é o produto, então sem DISTINCT a mesma seção apareceria centenas de vezes. Trouxe também a contagem de produtos por seção, que é o que dá dimensão ao resultado.

```
SELECT DISTINCT
  DEP_COD, DEP_NAME, SECTION_COD, SECTION_NAME
FROM data_product
WHERE DEP_NAME IN ('BEBIDAS', 'PADARIA')
ORDER BY DEP_NAME, SECTION_NAME
```

Dep. cód.	Departamento	Seção cód.	Seção	Produtos
2	BEBIDAS	4	BEBIDAS	1.038
2	BEBIDAS	29	CERVEJAS	215
2	BEBIDAS	31	REFRESCOS	23
2	BEBIDAS	30	VINHOS	522
7	PADARIA	8	DOCES-E-SOBREMESAS	734
7	PADARIA	27	GESTANTE	1
7	PADARIA	19	PADARIA	287
7	PADARIA	22	QUEIJOS-E-FRIOS	147

Achado de qualidade de dado: a seção **GESTANTE** aparece dentro do departamento **PADARIA** com um único produto. Pelo nome, é classificação de perfumaria ou farma, não de padaria. Um produto isolado não distorce relatório, mas o mesmo padrão numa base maior contamina análise de margem por seção. Vale checar na origem do cadastro.

2.3. Venda total de produtos por Business Area no primeiro trimestre de 2019

Usei data_product_sales, que é venda de produto, e não data_store_sales, que é o agregado da loja. A pergunta é sobre produto. O CAST resolve a diferença de tipo do STORE_CODE apontada na seção 1.2.

```

SELECT
  c.BUSINESS_CODE,
  c.BUSINESS_NAME
  AS BUSINESS_AREA,
  ROUND(SUM(s.SALES_VALUE), 2)
  AS VENDA_TOTAL_1T2019,
  SUM(s.SALES_QTY)
  AS QTD_TOTAL_1T2019,
  COUNT(DISTINCT s.STORE_CODE)
  AS QTD_LOJAS
FROM data_product_sales s
INNER JOIN data_store_cad c
  ON CAST(s.STORE_CODE AS UNSIGNED) = c.STORE_CODE
WHERE s.DATE BETWEEN '2019-01-01' AND '2019-03-31'
GROUP BY c.BUSINESS_CODE, c.BUSINESS_NAME
ORDER BY VENDA_TOTAL_1T2019 DESC

```

Cód.	Business Area	Venda total 1T2019 (R\$)	Quantidade	Lojas
4	Farma	81.776.691,73	2.854.179	4
1	Varejo	81.032.347,65	5.252.352	4
5	Atacado	80.384.884,60	5.221.672	4
2	Proximidade	80.171.122,80	5.209.730	4
3	Posto	32.072.326,40	2.347.032	4
TOTAL		355.437.373,18	20.884.965	20

Controle aplicado. Antes de aceitar o resultado, somei o trimestre inteiro sem nenhum join: **R\$ 355.437.373,18**. A soma das cinco áreas dá exatamente o mesmo valor, o que prova que o INNER JOIN não descartou nenhuma venda. Confirmei também que as 20 lojas que vendem existem nas duas tabelas e que nenhum STORE_CODE tem caractere não numérico.

Leitura: Farma lidera em faturamento com R\$ 81,8 milhões vendendo **2,85 milhões** de unidades, enquanto Varejo fatura praticamente o mesmo com **5,25 milhões** de unidades. É um ticket médio quase duas vezes maior. Posto é o piso das duas métricas. Vale registrar que Varejo, Atacado e Proximidade ficam muito próximos entre si, o que sugere base sintética gerada a partir de um mesmo padrão.

3. Case 1: função dinâmica de consulta

O problema descrito é o time reescrever a mesma query trocando só os filtros. A solução é transformar a query em função com contrato claro, para que a mudança de filtro deixe de ser edição de texto e passe a ser parâmetro. Como o enunciado avisa que outras equipes vão usar, tratei isso como código de biblioteca e não como script de uso único.

3.1. Premissas que assumi

1. **Todo filtro é opcional.** Quem chama sem argumento quer a base, e por isso existe uma trava de limite.
2. **Cada filtro aceita um valor ou uma lista.** "A loja 1" e "as lojas 1, 3 e 7" são a mesma pergunta.
3. **Date segue o enunciado:** lista ISO, com um item para dia exato e dois para intervalo fechado.
4. **Nenhum valor entra na SQL por concatenação.** Tudo vai como parâmetro vinculado.
5. **Erro tem que ser explícito.** Entrada inválida levanta exceção com mensagem, nunca retorna vazio.

3.2. Código

```
def _como_lista(valor) -> Optional[list]:
    """Aceita None, valor unico ou colecao, e devolve sempre lista ou None."""
    if valor is None:
        return None
    if isinstance(valor, (str, bytes)) or not isinstance(valor, Iterable):
        return [valor]
    lista = list(valor)
    return lista or None

def _valida_inteiros(valores, nome_campo) -> list:
    saida = []
    for v in valores:
        try:
            saida.append(int(v))
        except (TypeError, ValueError):
            raise ValueError(
                f"{nome_campo} precisa ser inteiro ou lista de inteiros. Recebi: {v!r}"
            )
    return saida

def _valida_datas(valores) -> list:
    """Aceita 'YYYY-MM-DD', date ou datetime. Devolve lista de strings ISO."""
    saida = []
    for v in valores:
        if isinstance(v, (_date, datetime)):
            saida.append(v.strftime("%Y-%m-%d"))
            continue
        try:
            saida.append(datetime.strptime(str(v).strip(), "%Y-%m-%d").strftime("%Y-%m-%d"))
        except ValueError:
            raise ValueError(
                f"date precisa estar no formato ISO 'YYYY-MM-DD'. Recebi: {v!r}"
            )
    if len(saida) > 2:
        raise ValueError(
            "date aceita no maximo 2 itens: ['dia'] para um dia ou "
            "['inicio', 'fim'] para um intervalo."
        )
    if len(saida) == 2 and saida[0] > saida[1]:
        # inverter em silencio esconderia erro de quem chama; melhor avisar e corrigir
        logger.warning("Datas invertidas (%s > %s). Reordenando.", saida[0], saida[1])
        saida = sorted(saida)
    return saida

# -----
# Funcao principal
# -----
def retrieve_data(
    product_code: Union[int, Sequence[int], None] = None,
    store_code: Union[int, Sequence[int], None] = None,
    date: Union[str, Sequence[str], None] = None,
    limit: Optional[int] = LIMITE_PADRAO,
    return_query: bool = False,
    engine: Optional[Engine] = None,
):
    """Traz as vendas de produto ja filtradas, em DataFrame.

    Parametros
    -----
    product_code : int | list[int] | None
       Codigo do produto. Aceita um ou varios. None traz todos.
    store_code : int | list[int] | None
       Codigo da loja. Aceita um ou varios. None traz todas.
    date : str | list[str] | None
       Data em ISO. ['2019-01-01'] filtra o dia,
        ['2019-01-01', '2019-01-31'] filtra o intervalo fechado. None traz tudo.
    limit : int | None
```

```

    Trava de segurança. Passe None para desligar quando precisar da base cheia.
return_query : bool
    True devolve (DataFrame, sql, parametros) para depurar ou auditar.
engine : Engine | None
    Permite injetar outra conexao, util em teste automatizado.

Retorno
-----
pandas.DataFrame com as colunas de data_product_sales.

Exemplos
-----
>>> retrieve_data(18, 1, ['2019-01-01', '2019-01-31'])
>>> retrieve_data(store_code=[1, 2, 3], date=['2019-12-25'])
>>> retrieve_data() # amostra geral, respeitando o limite
"""

filtros, parametros = [], {}

produtos = _como_lista(product_code)
if produtos:
    produtos = _valida_inteiros(produtos, "product_code")
    chaves = [f":prod_{i}" for i in range(len(produtos))]
    filtros.append(f"PRODUCT_CODE IN ({', '.join(chaves)})")
    parametros.update({f"prod_{i}": v for i, v in enumerate(produtos)})

lojas = _como_lista(store_code)
if lojas:
    lojas = _valida_inteiros(lojas, "store_code")
    chaves = [f":loja_{i}" for i in range(len(lojas))]
    # STORE_CODE eh varchar nesta tabela: comparo como numero dos dois lados
    # para nao depender de zero a esquerda nem de espaco no cadastro.
    filtros.append(f"CAST(STORE_CODE AS UNSIGNED) IN ({', '.join(chaves)})")
    parametros.update({f"loja_{i}": v for i, v in enumerate(lojas)})

datas = _como_lista(date)
if datas:
    datas = _valida_datas(datas)
    if len(datas) == 1:
        filtros.append("DATE = :data_ini")
        parametros["data_ini"] = datas[0]
    else:
        filtros.append("DATE BETWEEN :data_ini AND :data_fim")
        parametros["data_ini"], parametros["data_fim"] = datas

sql = f"SELECT {', '.join(COLUNAS)} FROM {TABELA}"
if filtros:
    sql += "\nWHERE " + "\n AND ".join(filtros)
sql += "\nORDER BY DATE, STORE_CODE, PRODUCT_CODE"
if limit:
    sql += f"\nLIMIT {int(limit)}"

logger.info("Executando: %s | parametros: %s", sql.replace("\n", " "), parametros)
eng = engine or get_engine()
df = pd.read_sql(text(sql), eng, params=parametros)

if limit and len(df) == limit:
    logger.warning(
        "0 retorno bateu no limite de %s linhas. Pode haver dado sobrando: "
        "aumente o limit ou passe limit=None.", limit
    )

if return_query:
    return df, sql, parametros
return df

```


3.3. Uso

```
my_data = retrieve_data(18, 1, ['2019-01-01', '2019-01-31']) # como no enunciado
retrieve_data(store_code=[1, 2, 3], date=['2019-12-25']) # varias lojas, um dia
retrieve_data(product_code=18) # so um filtro
retrieve_data(limit=None) # base inteira, sob demanda
```

3.4. Testes executados

#	Cenário	Resultado
1	Chamada do enunciado, produto 18, loja 1, janeiro de 2019	31 linhas, soma R\$ 29.124,80
2	Dia único: 25/12/2019 na loja 1	75 linhas
3	Produtos [18, 19] em lojas [1, 2, 3] no 1T2019	270 linhas
4	Só o filtro de produto	Trava de limite acionada e registrada em log
5	Sem filtro nenhum	Trava segurou as 2,1 milhões de linhas
6	Datas invertidas	Reordenou e avisou, em vez de devolver vazio
7	Data em formato brasileiro, 31/01/2019	ValueError com mensagem clara
8	store_code = 'loja um'	ValueError com mensagem clara
9	product_code = "18; DROP TABLE data_product_sales"	Barrado na validação, antes do banco
10	Função comparada com SQL escrito à mão	Mesmas 31 linhas e mesma soma

Por que o cenário 10 importa. Uma função que monta SQL sozinha só é confiável se produzir exatamente o mesmo resultado da query escrita manualmente. Essa comparação é o teste que fecha a questão.

4. Case 2: TM por loja no quarto trimestre de 2019

4.1. A instrução do cliente vem antes da otimização

O cliente foi explícito: usar as queries como estão, sem alterar nem criar novas, e filtrar o período indicado. A Query 2 traz 2019 inteiro, 7.300 linhas. Seria mais eficiente mudar o WHERE, mas isso descumpriria a única instrução dada. Executei as duas queries intocadas e apliquei o recorte no Python.

```
# Query 2 do cliente, sem uma virgula alterada
SELECT
    STORE_CODE,
    DATE,
    SALES_VALUE,
    SALES_QTY
FROM data_store_sales
WHERE DATE BETWEEN '2019-01-01' AND '2019-12-31'

# recorte pedido, aplicado depois, no pandas
vendas["DATE"] = pd.to_datetime(vendas["DATE"])
q4 = vendas[(vendas["DATE"] >= "2019-10-01") & (vendas["DATE"] <= "2019-12-31")]
# 7.300 linhas -> 1.840 linhas, 20 lojas, 01/10 a 31/12
```

Sugestão para uma próxima rodada, fora do escopo desta entrega: se o volume crescer, vale negociar com o cliente o filtro dentro da query, porque trazer o ano inteiro para descartar 75% em memória não escala.

4.2. O enunciado não define TM, então testei as duas leituras

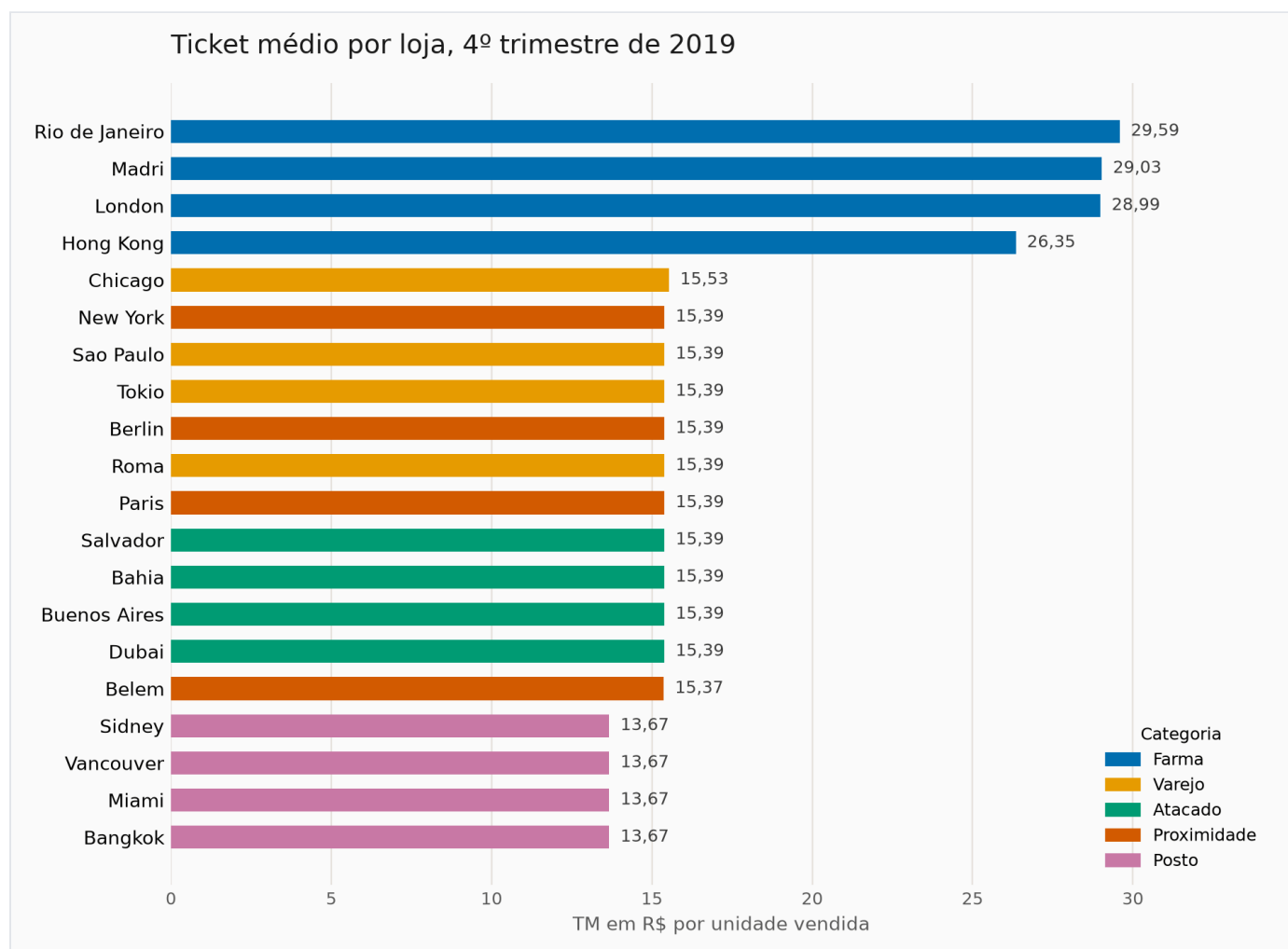
TM pode significar soma do valor dividida pela soma da quantidade, ou a média das razões diárias. Em vez de escolher por intuição, calculei as duas e comparei com a tabela de referência publicada no desafio.

```
q4["TM_LINHA"] = q4["SALES_VALUE"] / q4["SALES_QTY"]
agr = q4.groupby("STORE_CODE").agg(
    SALES_VALUE=("SALES_VALUE", "sum"),
    SALES_QTY=("SALES_QTY", "sum"),
    TM_MEDIA_DAS_MEDIAS=("TM_LINHA", "mean"),
).reset_index()
agr["TM_SOMA SOBRE SOMA"] = (agr["SALES_VALUE"] / agr["SALES_QTY"]).round(2)
```

Resultado da comparação: a fórmula **soma sobre soma** reproduz a tabela esperada em **20 de 20 lojas**. A média das médias erra por um centavo no Rio de Janeiro, 29,58 contra 29,59. Adotei a soma sobre soma, que além de reproduzir o resultado é a definição correta de ticket médio de um período.

4.3. Resultado

Loja	Categoria	TM	Loja	Categoria	TM
Bahia	Atacado	15,39	Miami	Posto	13,67
Bangkok	Posto	13,67	New York	Proximidade	15,39
Belem	Proximidade	15,37	Paris	Proximidade	15,39
Berlin	Proximidade	15,39	Rio de Janeiro	Farma	29,59
Buenos Aires	Atacado	15,39	Roma	Varejo	15,39
Chicago	Varejo	15,53	Salvador	Atacado	15,39
Dubai	Atacado	15,39	Sao Paulo	Varejo	15,39
Hong Kong	Farma	26,35	Sidney	Posto	13,67
London	Farma	28,99	Tokio	Varejo	15,39
Madri	Farma	29,03	Vancouver	Posto	13,67



Ticket médio por loja no 4º trimestre de 2019, cor por categoria.

4.4. Por que este gráfico

São 20 lojas nominais comparadas por uma única medida, ou seja, o trabalho do gráfico é **ranquear magnitude**. Barra é a forma que o olho compara com mais precisão, e na horizontal porque nomes como "Rio de Janeiro" e "Buenos Aires" ficariam inclinados num eixo vertical.

Ordenei por valor, e não alfabeticamente, porque a pergunta é sobre posição relativa. A cor identifica a categoria e usa uma paleta verificada para daltonismo, com o valor escrito na ponta de cada barra para que a leitura não dependa só da cor.

4.5. Leitura de negócio

- As quatro lojas **Farma** operam noutra patamar, de R\$ 26,35 a R\$ 29,59, quase o dobro do resto.
- **Varejo, Atacado e Proximidade** são praticamente idênticas em R\$ 15,39, e **Posto** é o piso, R\$ 13,67 nas quatro.
- A categoria explica o ticket melhor do que a loja individual. As únicas que fogem do próprio grupo são **Chicago**, com R\$ 15,53, e **Belém**, com R\$ 15,37.
- Consequência prática: comparar loja de Farma com loja de Posto no mesmo ranking induz erro. A comparação justa é dentro da categoria.

5. Case 3: visualização própria com IMDB_movies

5.1. Perfil da tabela antes de qualquer gráfico

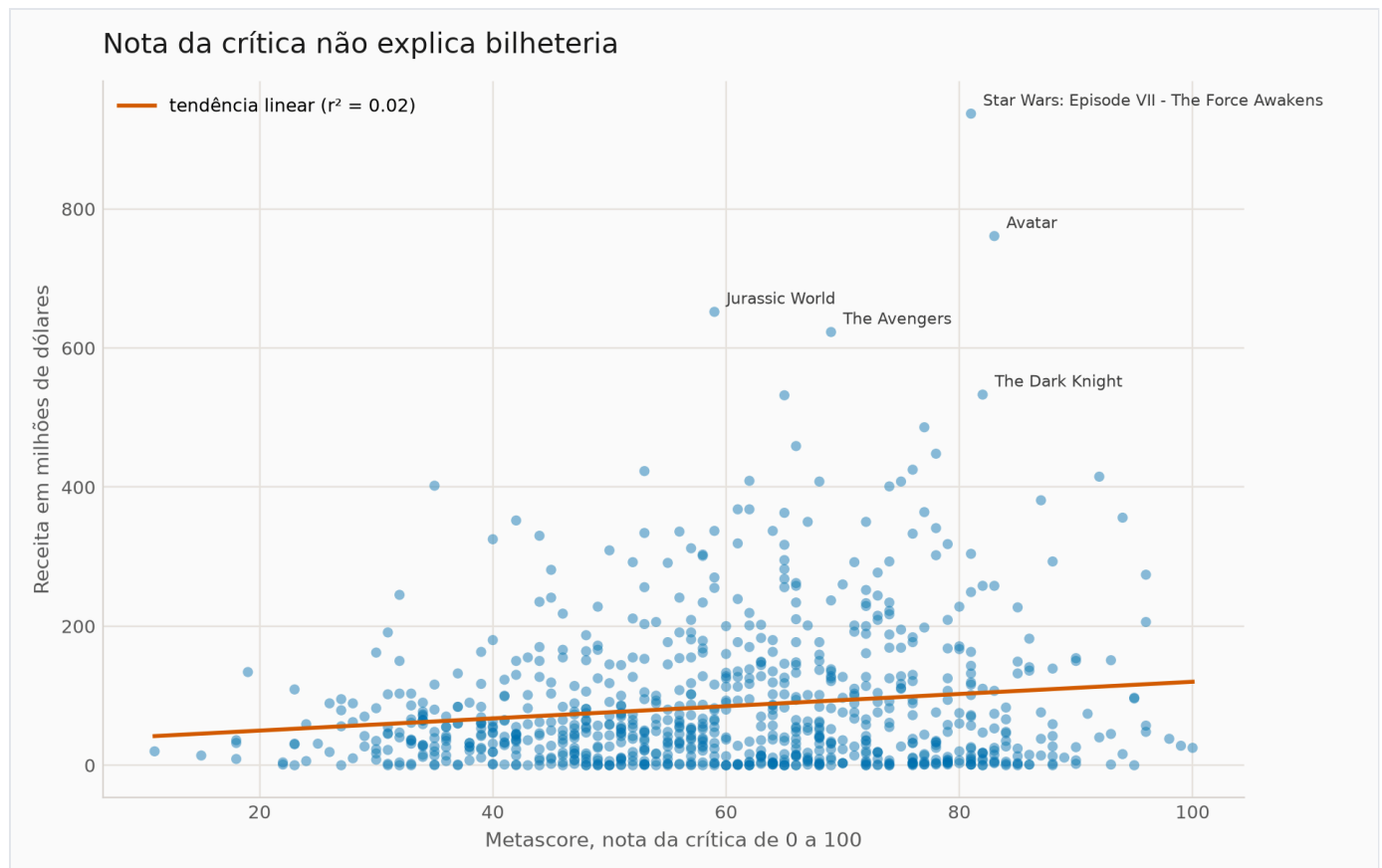
Coluna	Situação
Linhas	1.000 filmes, de 2006 a 2016
RevenueMillions	128 nulos , mediana US\$ 48 mi, máximo US\$ 937 mi
Metascore	64 nulos , de 11 a 100, mediana 59,5
Rating	Apenas 8 valores distintos: 2, 3, 4, 5, 6, 7, 8 e 9
Genre	Multivalorada: "Action,Adventure,Sci-Fi" numa única célula

Decisão técnica registrada: Rating e RevenueMillions estão tipados como decimal(10,0), ou seja, **sem casa decimal**. A nota 8,4 foi gravada como 8. A precisão se perdeu na modelagem da tabela, não no dado de origem. Por isso **não usei Rating como eixo contínuo**, já que sobram só 8 valores possíveis e o gráfico viraria uma escada. A análise usa **Metascore**, que é inteiro de 0 a 100 e preservou a granularidade.

5.2. Pergunta escolhida

Nota de crítico se converte em bilheteria, e quais gêneros concentram receita? É a pergunta que um estúdio faria antes de decidir onde colocar dinheiro.

5.3. Gráfico 1: crítica contra bilheteria



838 filmes com Metascore e receita informados. Os cinco maiores em bilheteria estão nomeados.

Por que dispersão: a pergunta é sobre a **relação entre duas variáveis contínuas**, e dispersão é a forma que mostra relação, inclusive quando ela não existe. A linha de tendência e o r^2 dão a medida objetiva, e nomear os cinco campeões de bilheteria transforma o argumento em algo verificável.

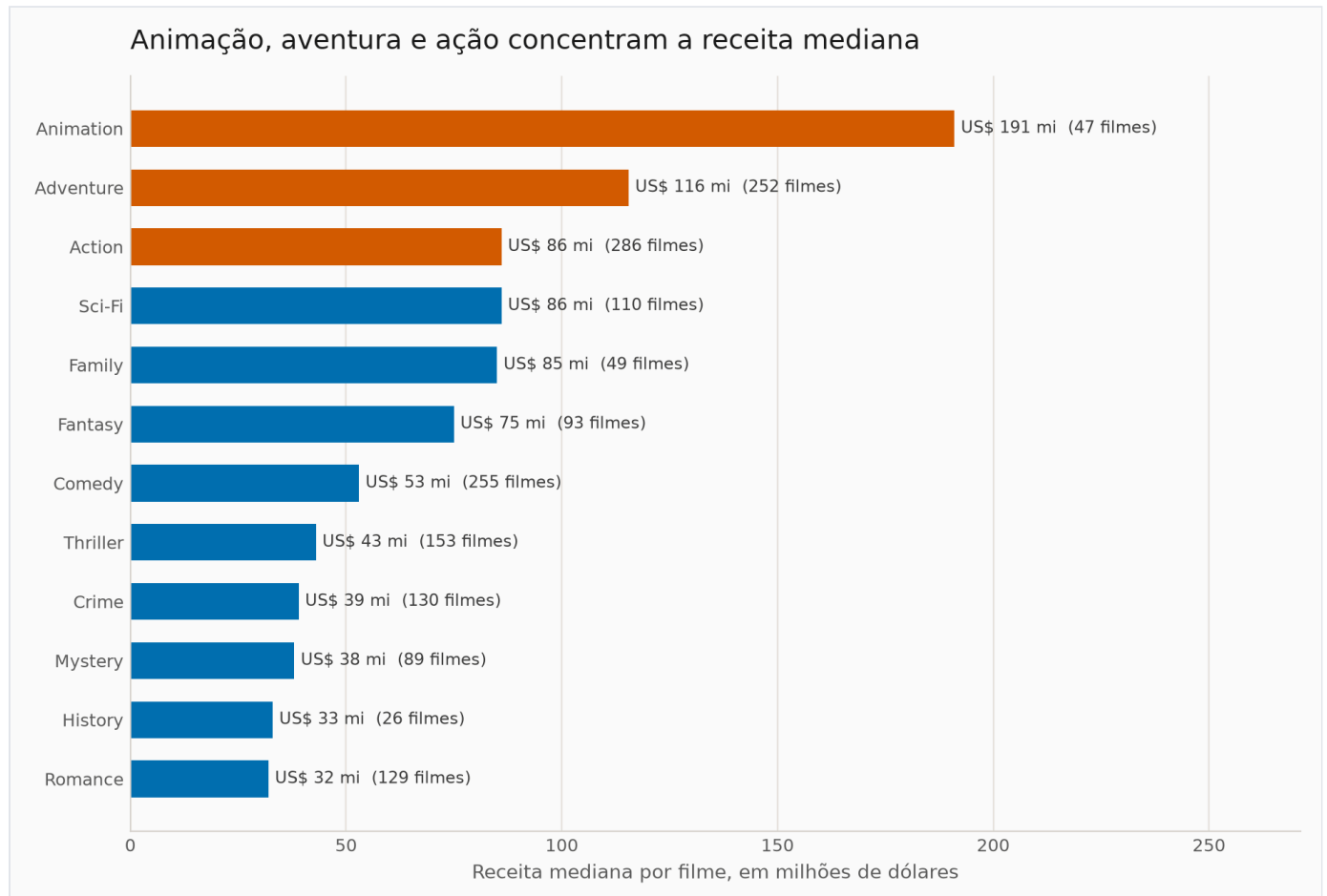
O que o dado responde: a correlação é de apenas $r = 0,142$, com $r^2 = 0,02$. A nota da crítica explica cerca de **2%** da variação de bilheteria. Mais forte ainda: filmes com Metascore de 80 ou mais têm receita mediana de **US\$ 41 mi**, enquanto os com menos de 50 têm **US\$ 43,5 mi**. Os mal avaliados faturam um pouco mais na mediana. Já o número de votos, que é medida de popularidade e não de qualidade, tem correlação de **0,637** com a receita.

Conclusão para negócio: se o objetivo é bilheteria, alcance e público importam muito mais que aprovação da crítica. Nota alta serve a outro objetivo, como prêmio e prestígio, e os dois não devem ser medidos pelo mesmo indicador.

5.4. Gráfico 2: receita mediana por gênero

```
# Genre eh multivalorada, entao explodo antes de agrupar,
# senao "Action,Adventure,Sci-Fi" viraria uma categoria falsa
g = df.assign(Genre=df["Genre"].str.split(",")).explode("Genre")
g["Genre"] = g["Genre"].str.strip()

resumo = (g.dropna(subset=["RevenueMillions"])
        .groupby("Genre")
        .agg(filmes=("Id", "count"),
             receita_mediana=("RevenueMillions", "median"))
        .query("filmes >= 20")
        .sort_values("receita_mediana", ascending=False))
```



Gêneros com pelo menos 20 filmes. A quantidade de filmes aparece ao lado do valor.

Duas decisões que mudam o resultado: primeiro, **explodir** a coluna de gênero, porque cada filme tem até três e tratar a combinação como categoria criaria dezenas de grupos artificiais. Segundo, usar **mediana** em vez de média, porque um Avatar sozinho distorce a média de um gênero inteiro. Também exige um mínimo de 20 filmes por gênero, para que nenhuma barra represente amostra pequena demais, e deixei a contagem visível ao lado de cada valor.

O que o dado responde: **Animação** tem receita mediana de **US\$ 191 mi**, quase o dobro de Aventura, com US\$ 116 mi, e mais de sete vezes a de Drama, com US\$ 27 mi. Drama é o gênero mais numeroso da base, 440 filmes, e um dos que menos faturam por filme. Volume de produção e retorno por filme são coisas diferentes.

6. Observações consolidadas sobre a base

Onde	O que encontrei	Impacto
data_product_sales x data_store_cad	STORE_CODE como varchar de um lado e int do outro	Join sem CAST pode perder venda em silêncio
data_store_cad	START_DATE e END_DATE gravadas como text	Comparação de data vira comparação de string
data_product	Seção GESTANTE dentro do departamento PADARIA	Classificação incorreta afeta análise por seção
data_product	Empate de preço na fronteira do top 10	Corte por LIMIT esconde item empatado
IMDB_movies	Rating e RevenueMillions como decimal(10,0)	Perda de precisão já na modelagem
IMDB_movies	128 nulos em receita e 64 em Metascore	Exige recorte explícito antes de correlacionar

7. Declaração de uso de ferramentas de IA

Conforme solicitado no e-mail do processo seletivo, declaro de forma aberta onde houve apoio de ferramenta de IA na construção desta solução e onde o trabalho foi meu.

Ferramenta utilizada

Claude (Anthropic), usado de forma pontual, em dois momentos específicos: conferência de trechos que eu já tinha escrito e executado, para checar se estavam corretos, e montagem da versão final deste documento. Não foi usado como par de programação contínuo, e a solução não foi construída dentro da ferramenta.

Como o trabalho foi conduzido

Conduzi o desafio em etapas, na sequência descrita na seção 1. Escrevi os scripts, executei no meu próprio ambiente contra o banco da Looqbox, li a saída, validei o resultado e só então segui para a etapa seguinte. Quando quis uma segunda opinião sobre um ponto específico, levei o trecho já pronto à ferramenta e pedi que apontasse erro, e a decisão de aceitar ou descartar continuou sendo minha. Nenhum número deste documento foi aceito sem execução real. **Escrevi e revisei o código linha a linha e assumo integralmente cada decisão técnica presente nesta entrega.**

Onde a ferramenta apoiou de forma direta

- Conferência pontual de trechos que eu já tinha escrito e rodado, para verificar se a lógica e a sintaxe estavam corretas.
- Redação e formatação deste documento, a partir das anotações, saídas e conclusões que eu produzi.

Onde está o meu domínio técnico

- Escrita dos scripts em Python, incluindo tratamento de exceção, logging e organização em módulo.

- Diagnóstico do erro 1044 como divergência de nome de schema, e não credencial inválida.
- Decisão de mapear tipos, volumes e período antes de escrever qualquer query.
- Escolha de data_product_sales em vez de data_store_sales na questão 3, por granularidade.
- Exigência de um número de controle para provar que o INNER JOIN não perdeu venda.
- Uso de CAST na chave por causa da divergência varchar contra int, e a razão de isso importar.
- Uso de parâmetro vinculado em vez de concatenação de string na montagem da query.
- Decisão de testar as duas fórmulas de TM contra a tabela de referência em vez de escolher por intuição.
- Decisão de descartar Rating como eixo contínuo depois de identificar a perda de precisão no tipo.
- Escolha das formas de visualização e toda a interpretação de negócio apresentada.

Onde precisei pesquisar

- Comportamento do CAST na comparação entre varchar e int no MySQL.
- Uso de DENSE_RANK como alternativa ao LIMIT em caso de empate.
- Ajustes finos de layout no matplotlib.

Sustento e explico qualquer trecho desta entrega ao vivo, na hora que a equipe quiser, inclusive reescrevendo qualquer parte do código durante a conversa.

Entendo que essa é a verificação mais honesta do que está declarado aqui, e me coloco à disposição para ela.